

Nutsolver: Computing Optimal Poker Strategies

Brian Tay

May 22, 2026

Abstract

This project presents the design and implementation of a solver that calculates the game theory optimal strategy in No-Limit Texas Hold'em Poker. Unlike perfect information games like chess where everything about the current game state is known, the two cards of the opponent are hidden in poker. However, a Nash equilibrium spot is known to exist when there are two players, meaning that there is an optimal, unexploitable strategy of playing. The goal of this project is to develop an open-source solver that can accurately and efficiently calculate said strategy.

Poker can be represented as a game tree, where each node contains information on the current game state, and a player's strategy in a given spot. In each leaf node, some player wins the pot; from there, the expected value of each previous node can be calculated by backtracking. Chance-sampling Monte Carlo Counterfactual Regret Minimization (CFR) is used to calculate Nash equilibrium by running two agents against each other that aim to exploit each other's strategy until equilibrium is reached.

Currently, the solver is able to calculate the optimal solution for any spot in poker starting with five community cards and exactly two players in the pot. Given the stack size, pot size, board cards, player ranges, and bet/raise sizes, the solver is able to calculate the optimal Nash equilibrium strategy. Future enhancements include upgrading the solver so that it's able to solve starting from earlier betting rounds, implementing other Monte Carlo CFR algorithms, and adding features like nodelocking to be able to better analyze exploitative strategies.

1 Introduction

No-Limit Texas Hold'em is the most popular version of poker. In this game, each player is dealt two private cards, known as "hole cards," while five shared community cards are gradually placed face-up on the table over multiple betting rounds. Players attempt to form the best possible five-card hand using any combination of their hole cards and the community cards.

The game consists of four betting rounds, called the preflop, flop, turn, and river. The preflop

has zero community cards, the flop has three, the turn has four, and the river has all five. During each betting round, players may choose among several actions. Betting is when a player is the first one in the betting round to put chips into the pot. In response, a player can fold (give up), call (match the bet), or raise (put even more chips into the pot). If no bets have been made, players can also check (pass their turn). The betting round ends when all players still in the pot have either checked or put the same amount of chips into the pot. A player wins all of the chips in the pot if everyone else folds or if they have the best five-card hand at showdown (after the last betting round).

In No-Limit Texas Hold'em, players may bet any amount of their remaining chips at any time, greatly increasing the number of possible decision branches. Because players cannot see their opponents' private cards, the game is classified as an imperfect-information game. This hidden information increases the complexity and makes finding an optimal strategy difficult. Unlike games such as chess, where all information is visible to both players, poker requires players to make decisions without knowing their opponents' private cards.

Modern poker solvers attempt to compute a Nash equilibrium strategy, which is a stable game-theoretic state in which no player can improve their expected outcome by independently changing strategies [1]. This is known as game theory optimal (GTO). These solvers have become widely used by professional poker players, researchers, and AI developers. However, most commercially available poker solvers, such as PioSOLVER [2], are expensive and proprietary. Although these programs rely on algorithms that are publicly described in academic literature, their implementations remain inaccessible.

The purpose of this project was to design and implement an open-source poker solver capable of approximating optimal strategies for simplified Texas Hold'em river situations. The project specifically focused on demonstrating that advanced equilibrium-solving techniques could be reproduced using publicly available algorithms and standard programming tools.

The final project successfully generated equilibrium approximations for two-player poker spots starting at the river and produced readable visual outputs showing optimal betting frequencies. This project contributes an accessible educational framework for studying computational game theory and demonstrates the feasibility of creating open-source alternatives to commercial poker solvers.

2 Procedure

The project consisted of designing four major components. There is a C++ poker-solving engine which contains the solving algorithm, a C++ poker hand evaluator, and a graphical user interface made in Python to neatly visualize the solver outputs. A flask app was also made as a centralized way for someone to use the solver.

2.1 Software and Environment

All source code is available at <https://github.com/guineapig25/poker-solver>.

- Programming Language: C++ & Python

- Compiler: GNU g++
- GUI Framework: Tkinter, Matplotlib, Flask
- Operating System: Linux / WSL environment

2.2 Input Data

In a file named ranges.txt, there are nine parameters the user sets that define the game tree generated by the solver. The first two parameters are the player ranges; these are all the possible cards that the out-of-position (OOP, first to act) player and the in-position (IP, second to act) player can have. The third parameter holds the five community cards, since the solver starts solving at the river. The fourth parameter is the pot size, and the fifth parameter is the effective stack size (the smaller stack size of the two players). The last four parameters are the possible bet and raise sizes that the OOP and IP players are allowed to perform, defined with integers that indicate betting/raising that percentage of the pot.

2.3 Game Tree Construction

The solver models poker as a decision tree [3] in which each node represents a possible unique scenario that could occur in the game and each edge represents a legal player action (checking, betting, etc.). Leaf nodes represent folds or showdowns, where a player wins the pot and the game ends. Recursion is used to generate all reachable states within the defined scenario.

In each node, the solver stores the current player, pot size, remaining stack sizes, and the previous betting action. The solver also stores what it thinks the optimal strategy for the current player is at that node. For each legal action, there is a decimal that represents the frequency at which that action is taken.

2.4 Algorithm

To solve the game space, Chance-Sampling Monte Carlo Counterfactual Regret Minimization [4] is used. In this algorithm, two agents repeatedly change their strategy to exploit the other player's strategy. It works as such:

1. Each node's action frequency is originally uniformly random
2. Repeat until maximum iteration count is reached:
 - a. Randomly sample a possible set of hole cards from each player's range
 - b. Backtrack from terminal nodes for those cards to calculate expected value (EV) at each node
 - c. Compute regret of each action at each node: $\text{Regret} = \text{Action EV} - \text{Strategy EV}$
 - d. Set frequency of negative regret actions to be zero
 - e. Set frequency of positive regret actions using the formula $\text{Regret} / \text{Sum of Positive Regrets}$

2.5 Evaluation

Initially, toy poker games were used to evaluate the solver, which are simple game parameters where the exact equilibrium solution is already known. Kuhn poker was tested, which is a toy poker game where the deck only has three cards (A, K, and Q), and the highest card wins [5]. Shown in Figure 4, the solver successfully found one of the many possible Nash equilibriums that exist in Kuhn poker.

Nash distance was also a method used to evaluate the solver’s ability to reach Nash equilibrium. Nash equilibrium means that a strategy is not exploitable; hence, Nash distance (the distance from equilibrium) refers to the amount, given as a percentage of the pot, that a player’s strategy can be exploited by the other player. This metric can be computed by averaging the positive regrets of a player and dividing it by the size of the pot. The iteration v. Nash distance curve (shown in Figure 5) shows how the Nash distance successfully decreased down to 0.1% of the pot as more iterations were run. This is expected, since strategies become less exploitable as they are optimized throughout many iterations.

3 Results

3.1 Figures and Visualizations

Figure 1 is the ranges.txt file, which is set by the user and defines the game tree to the solver. The first two parameters are the possible hands that the OOP and IP player can have. These are defined by two letters representing the two hole cards. If the two cards are different, there must be an s afterward if the cards are the same suit or an o if the cards are offsuit. In the board parameter, five cards must be listed, denoted by the card’s value and the suit (s for spade, d for diamond, c for club, and h for heart). The pot and the stack parameters are represented as the exact number of chips there are in each. The last four parameters, for bet and raise sizing, contain several numbers that show possible bet sizes as a percentage of the current pot. For example, the OOP player is allowed to bet 25% or 100% of the pot in this scenario. In addition, checking will always be a possible action if it is legal, and it does not need to be specified.

```

≡ ranges.txt
1  OOP: AA KK QQ JJ AKs AKo AQs KQs
2  IP:  AA KK QQ JJ AKs AKo AQs KQs
3  BOARD: 2s 4d 2c 3s 4s
4  POT: 10
5  STACK: 100
6  OOP_BET_SIZES: 25 100
7  OOP_RAISE_SIZES: 50 100
8  IP_BET_SIZES: 25 50 75 100
9  IP_RAISE_SIZES: 25 50 75 100

```

Figure 1: Input data for a sample simulation.

Two nodes of the solver results for the simulation with the parameters in Figure 1 are shown in Figure 2 and Figure 3. In total, approximately 23,000 iterations were run, solving to the accuracy of 0.1% of the pot. At the top of the UI, there is a node selector that allows the user to view the optimal strategy at each node in the game tree. The left side has a poker chart of each possible private hand a player can have. Hands in the player's range are shaded certain colors based on the frequency of the actions that hand should take. Green represents checking/calling, light blue represents folding, and red represents betting. Darker shades of red represent bigger bet sizing. When clicking on a cell in the game tree, the exact frequencies for each action that the hand should take are displayed on the right side.

Figure 2 shows the OOP player's strategy at the root node, which is the very first turn when no players have acted yet. The solver wants them to check their entire range, which is a very common solver strategy because the OOP player is at an information disadvantage due to going first. The 0.01 frequency action of betting 25% of the pot with AA can likely be disregarded as solver noise, since not enough iterations were run so the solver did not reach perfect accuracy.

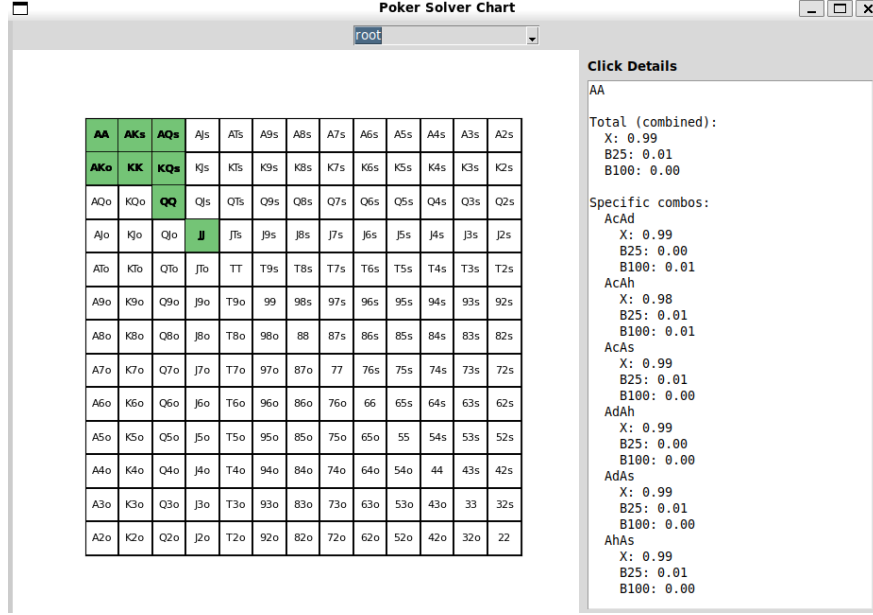


Figure 2: Solver results at the root node for a sample simulation.

Figure 3 shows the IP player's strategy when the OOP player checks. It plays a pretty balanced strategy, mixing bets and checks which is logical for a general solver strategy since it is trying to be unpredictable.

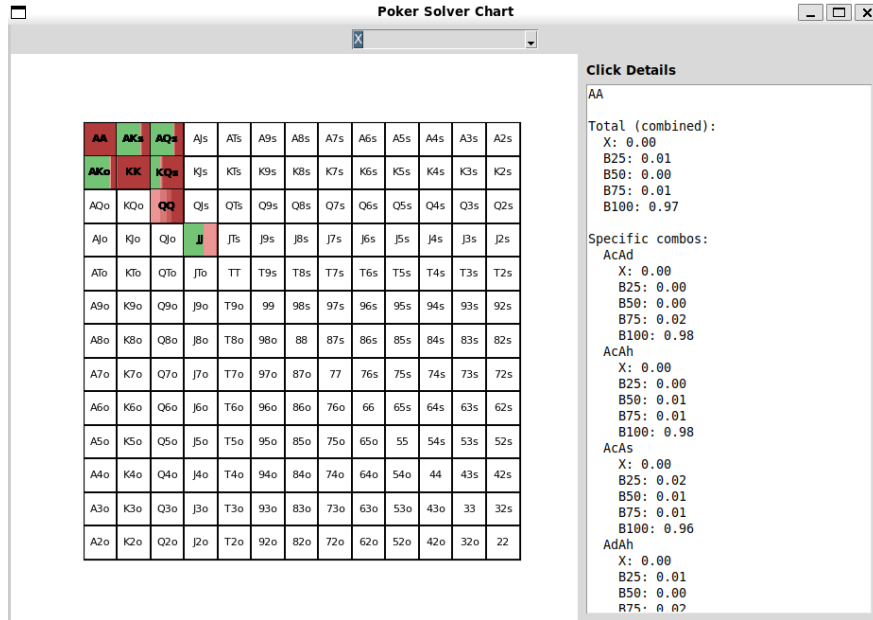


Figure 3: Solver results at the X (check) node for a sample simulation.

Figure 4 shows that the solver correctly calculated one of the possible Nash equilibriums for Kuhn Poker. In response to the OOP player checking, the IP player should be betting AA all the time, checking KK all the time, and betting QQ 33% of the time.

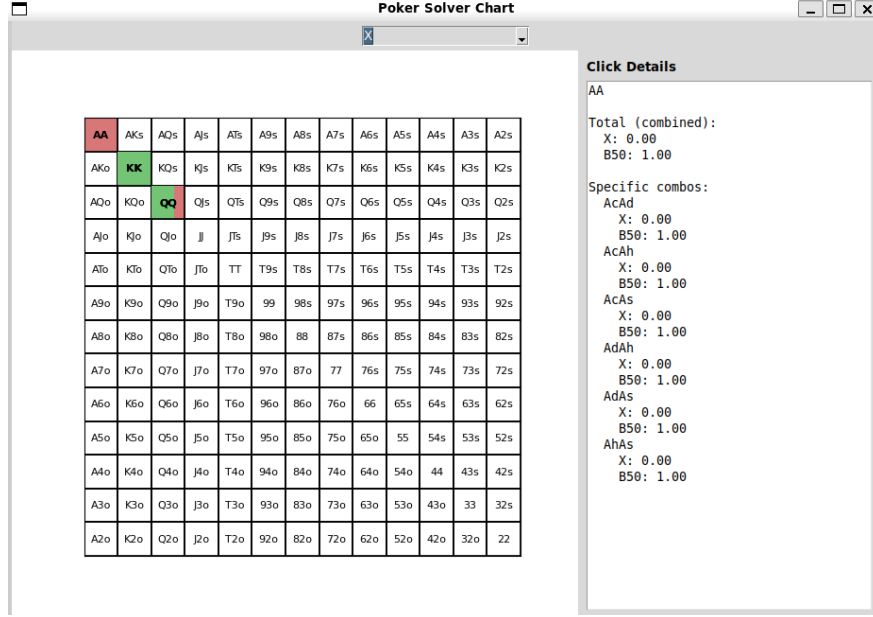


Figure 4: Solver results at the X node for Kuhn Poker.

3.2 Training Curves

The Nash distance curve in Figure 5 provides evidence that the solver is converging to equilibrium as expected. The Nash distance was strictly decreasing as more iterations were run, which shows that each player's strategy became less and less exploitable. The final exploitability of 0.1% of the pot means that it is impossible to win more than an extra 0.1% of the starting pot size against the final solver strategy. This accuracy is considered good enough because it is already well beyond human levels of play.

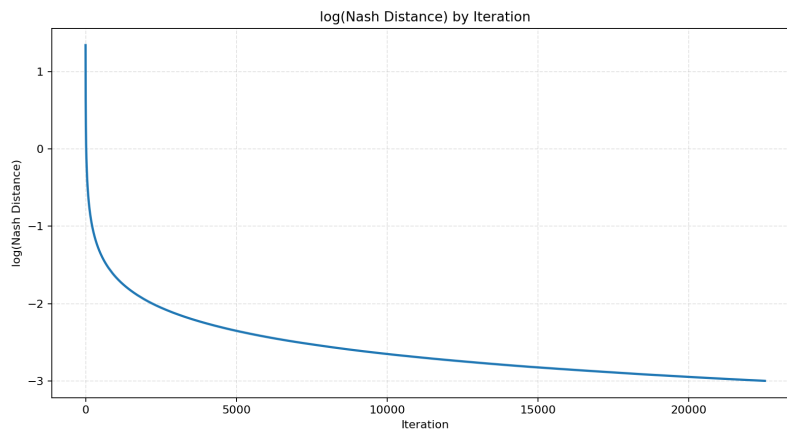


Figure 5: Solver simulation up to an exploitability of 0.1% of the pot.

3.3 Final Product

This app, made with Flask, centralizes the key components of the solver to make it easy to use. The ranges.txt file is at the top of the app, which the user can modify. The iteration count can also be changed, and the solver can be run with a single button. After the solver is finished, the results UI will pop up automatically. There is also the console log at the bottom of the app, which shows the direct output and solver results in a text format.

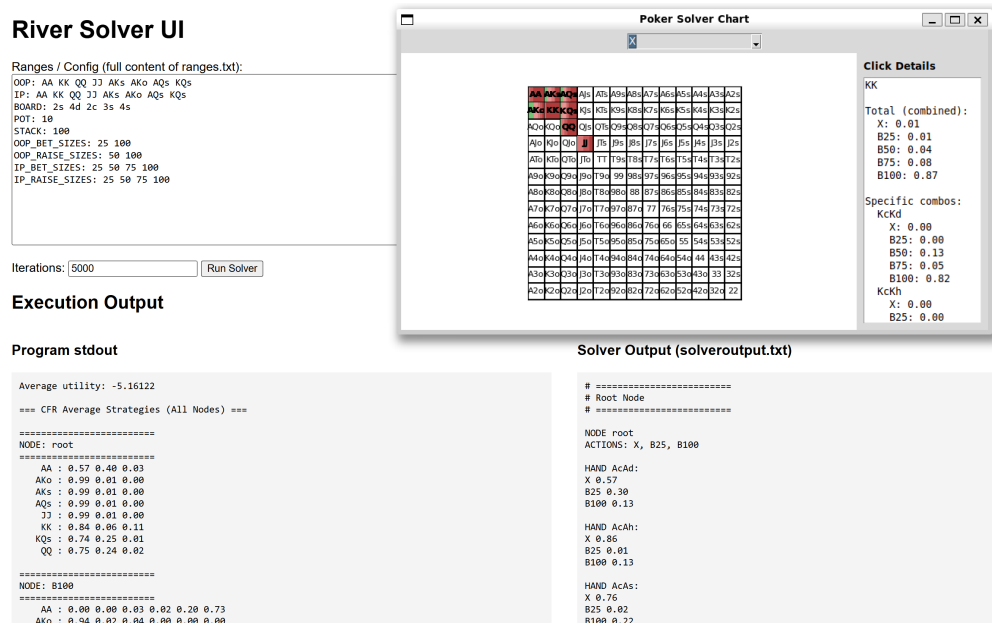


Figure 6: Screenshot of the Flask app for the poker solver.

4 Conclusions

This project successfully developed an open-source poker solver capable of approximating equilibrium strategies for No-limit Hold'em river scenarios. By combining recursive game tree construction with Chance-Sampling Monte Carlo Counterfactual Regret Minimization (CFR), the solver computed stable mixed strategies consistent with principles of poker theory.

The implementation demonstrated that advanced poker-solving techniques used in commercial software, such as PioSOLVER, can be recreated using publicly available algorithms and open-source tools. The use of C++ provided efficient computation for large decision trees, while Python enabled clear visualization of equilibrium outputs.

Although the solver currently focuses on simplified river situations, the project establishes a strong foundation for future development. Current limitations include restricted betting abstractions, limited game-tree size, and simplified scenario modeling compared to professional commercial solvers.

Future work may include expanding to solving from previous betting rounds, since only being able to solve from spots starting on the river is a limiting factor for the practical use of

the solver for studying and hand analysis. In addition, runtime optimizations that sacrifice some accuracy for a large speedup could be included as a feature for large game trees. The number of nodes in the game tree gets exponentially bigger as more parameters are defined, so runtime becomes a more important factor.

Overall, this project demonstrates the accessibility and educational value of open-source computational game theory tools and contributes toward broader public access to poker-solving technology.

References

- [1] Tom Boshoff, “GTO Wizard: What does GTO aim to achieve?” 2022. Available: <https://blog.gtowizard.com/what-does-gto-aim-to-achieve/>
- [2] PioSOLVER Team, “PioSOLVER: A very fast GTO solver for holdem.” 2026. Available: <https://piosolver.com/>
- [3] Tom Boshoff, “GTO Wizard: How solvers work.” 2023. Available: <https://blog.gtowizard.com/how-solvers-work/>
- [4] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, “Regret minimization in games with incomplete information,” *Advances in neural information processing systems*, vol. 20, 2007.
- [5] M. I. Lorient and J. C. Diez, “Perfect bayesian equilibrium in kuhn poker,” Universidad de San Andres, Departamento de Economia, 2023.